LedgerOps

从一笔购买读懂后端账本

一份不用找按钮、从头顺着读的中文教程

从客户余额100开始，把购买、幂等、并发、冲正、核对和重建串成一个故事。每一步先解释问题，再读代码，推演结果全部写出。

适合：能读基础Python，正在补SQL、事务与并发。

代码参照：a08f80c · R1 / R2

正文可独立阅读，无需启动网页或数据库。

阅读路线

从开头顺读即可。目录和PDF书签可用于第二遍查找；第一次阅读不要求跳转源码、安装工具或操作网页。

写在前面：这次只跟着一条故事走 3

第一章：这个项目到底在做什么 3

第二章：钱怎样记下来，为什么要存两份表示 5

第三章：先读懂数据库和SQL，再去看业务函数 7

第四章：一笔购买怎样完整地走过Python和数据库 9

第五章：请求重来一次，为什么不会再扣80 11

第六章：两笔80同时花100，事务为什么还不够 13

第七章：冲正怎样把钱还回去，同时保留历史 16

第八章：账都平了，为什么还可能把钱记错 17

第九章：把120修回100，为什么要先冻结 20

第十章：现在去读代码，应该先看哪几段 23

附录A：面试继续往下追时，怎样展开 25

附录B：源码地图与几个容易忘的词 26

一份可以从头顺着读的中文教程

适合：能读基础 Python，希望把 SQL、事务、并发和项目代码串起来的你。

依据：LedgerOps 本地提交 `a08f80c`，R1 / R2 与学习实验室。编写日期：2026-10-28。

写在前面：这次只跟着一条故事走

这份教程把过程、结果、代码和原因都写在正文里。你从第一章往下读，就能跟完一笔钱的完整经历；网页只是以后想亲手验证时的可选工具。

我们始终用同一组数字：客户最初有100，向商户买80；再次买80会被拒绝；商户把第一笔购买冲正，客户恢复100；接着故意把余额弄错成120，最后发现错误并修回100。并发问题另用一个明确重开的例子讲，两笔80同时竞争最初的100。

这里的100、80都是最小货币单位。项目使用PLN，100 grosz等于1.00 PLN。正文写“余额100”，指100个最小单位，并非100 PLN。开户资金来自合成测试夹具，项目没有接真实银行充值。

第一次阅读，先跟住数字、表和执行顺序。函数名会在需要时出现，并附上它负责的事情。最后两章把这些知识连到源码、测试和面试表达。理解分两层：先能解释为什么这样记，再能沿着代码指出它在哪里完成。第一遍先抓住每个函数解决的问题，再回头读它的具体语句。

文中的M1、W、P1、K1都是便于阅读的别名。实际代码使用UUID或有作用域的请求键。SQL中标为“教学简化”的片段用于解释含义，省略了真实实现的部分字段和检查；标为“源码节选”的片段保留项目中的具体写法。

第一章：这个项目到底在做什么

1.1 先想清楚“客户余额”是什么

假设一个平台替客户保管资金。客户显示有100，意思是平台欠客户100。客户向商户购买80后，平台欠客户的变成20，欠商户的变成80。

这笔购买首先改变的是“平台欠谁多少钱”。它没有自动触发银行转账，也不证明商户已经从银行收到80。

时刻	平台欠客户	平台欠商户M1	平台清算资产
合成开户后	100	0	100
购买80后	20	80	100

所以，项目不是一个“把两个数字相减相加”的练习。它要保证：成功购买时这两边一起发生；重复请求不会再记一次；两个并发请求不能各自花掉同一份钱；发生故障后，还能检查和恢复保存下来的账。

1.2 为什么只写两条UPDATE还不够

先看一个容易想到的写法。下面是故意简化的伪代码，不是项目实现：

PYTHON

```
customer_balance -= 80
merchant_balance += 80
```

它只表达了理想结果，没有回答四件事：第一行做完程序崩溃怎么办？同一请求收到两次怎么办？两个请求同时认为余额够怎么办？代码把80加给了错误商户怎么办？

LedgerOps围绕这些失败情况组织代码。R1实现购买、业务身份、重放、并发扣款和事务重试；R2增加独立核对、冻结重建和全额冲正。它们共同构成一个本地账本核心。

1.3 当前项目有哪些层

把它看成四层，后面就不容易迷路。

层	这里负责什么	主要位置
输入与身份	金额格式、业务ID、账户ID、规范化请求	`src/ledgerops/domain/`
业务流程	授权、事务、认领身份、锁余额、保存结果	`src/ledgerops/db/admission.py`
数据库规则	表、键、权限、CHECK、历史保护与提交检查	`src/ledgerops/db/sql/`
独立核对与维护	重新计算、检查对象、导出、冻结和修复	`verification.py`、`export.py`、`db/maintenance.py`

Python通过psycopg向PostgreSQL执行显式SQL。项目没有使用ORM。`lab/`里的HTTP服务和页面是课堂适配器，正式账本包没有生产HTTP登录接口。`AdmissionResult.http_status`里的201、422等是核心返回的状态语义；学习页面接收正常业务结果时，外层HTTP请求可以是200，再在结果中显示201或422。

这一区别在面试中很重要：你可以讲清账本核心的行为，但不能把课堂固定身份说成生产认证，或把合成入金说成支付渠道集成。

第二章：钱怎样记下来，为什么要存两份表示

2.1 借方和贷方，不等于扣钱和加钱

借方记作D，贷方记作C。它们是记账方向，具体是增加还是减少，要先看账户类型。项目的观察视角始终是平台。

账户类型	代表什么	借方D	贷方C
清算资产	平台持有的清算资产	增加	减少
客户负债	平台欠客户的钱	减少	增加
商户负债	平台欠商户的钱	减少	增加

开户记“D清算100，C客户100”，两边都增加。购买记“D客户80，C商户80”，客户减少、商户增加。方向相同，含义却取决于资产还是负债。

这一点一旦明白，后面就不用死背“客户为什么借记”。因为客户余额是平台的负债，借记负债就是平台欠客户的减少。

2.2 一笔业务、一张凭证、两条分录行

本文统一用下面三个名字，避免都叫“交易”而混在一起。

`operation`是业务，例如“购买P1”。`journal`是一张会计凭证，声明这次记账应该有几行、借贷各是多少。`journal_entry`是凭证里的具体分录行。

购买80的记录可以读成：

层次	记录内容
业务operation	P1是一笔已接受的购买，结果201
凭证journal	J1属于P1，声明2行、单边总额80
分录行1	J1，客户账户，D，80
分录行2	J1，商户M1，C，80

`total_minor=80`的意思是借方合计80，同时贷方合计80。不是把两侧相加后得到160。

为什么要有表头声明？因为“借贷一样多”只说明平衡，不说明这张凭证究竟应该包含哪些行。表头声明和提交检查一起使用，才能拒绝事后再追加一对看似平衡的分录。

2.3 分录是历史，余额是从历史算出的投影

每次扣款都扫描全部历史会很慢，所以项目另外保存`account_balance`。它保存账户的累计借方`debits`、累计贷方`credits`、预留`reserved`、行版本`version`，并生成账面余额`booked`。

对客户和商户负债账户：

推演 / 结构

```
booked = credits − debits
available = booked − reserved
```

对清算资产账户：

推演 / 结构

```
booked = debits − credits
```

`available`表示当前还可用于这次支出的余额。`reserved`表示已被预留占用的金额。主线例子的预留一直是0；R2没有实现预留的完整业务生命周期。

下面把客户累计列完整算一次：

时刻	累计借方debits	累计贷方credits	账面余额booked
开户后	0	100	100
购买80后	80	100	20
全额冲正后	80	180	100

冲正恢复了净余额，但没有把累计值退回开户状态。购买发生过，之后又发生了冲正，这两件事都留在历史里。

2.4 为什么数据库生成余额，仍可能算错账

`booked`是生成列，调用方不能直接给它赋值。代码更新累计借贷，数据库再按公式生成余额。

这能防止“累计借贷没变，直接把booked填成另一个数”。但如果有人错误地把`credits`多加20，生成出来的`booked`也会多20。行内公式依然正确，整本账却已经偏离分录。

因此，两份表示各有作用：余额投影便于日常快速判断；完整分录提供独立重新计算的依据。后面的核对A，专门检查这两份是否一致。

第三章：先读懂数据库和SQL，再去看业务函数

3.1 表、行、列：就是有规则的数据集合

一张表保存一类记录。行是一个实例，列是这个实例的属性。账户表的一行表示一个账户；余额表的一行表示这个账户当前的累计借贷和预留。

主键标识表内唯一的一行。项目许多主键带`workspace_id`：同一个工作区内不能重复，不应省略作用域直接说“全世界唯一”。外键要求引用对象存在；它只能证明“这个账户存在”，不能证明“它就是请求想付钱的那个商户”。

唯一约束用于禁止重复键。CHECK检查可表达的值域关系，例如余额不能小于预留。触发器在数据库内执行额外规则；延迟约束触发器可以等到提交时再检查最终状态。

先记住这些规则各管一件具体事。约束存在，不等于所有业务含义都被证明。

3.2 SELECT：先找表，再找筛选条件

下面是教学简化SQL，用来读取客户余额：

SQL

```
SELECT booked, reserved
FROM ledger.account_balance
WHERE workspace_id = %s AND account_id = %s;
```

从上往下读：取`booked`和`reserved`两列；来自`ledger`命名空间里的`account_balance`表；只要指定工作区、指定账户的那一行。

`%s`是绑定参数的位置。Python把SQL和参数分开交给驱动，参数不是通过字符串拼接变成SQL语法。项目中的实际执行形式类似：

PYTHON

```
row = cur.execute(
    "SELECT booked, reserved FROM ledger.account_balance "
    "WHERE workspace_id=%s AND account_id=%s",
    (workspace_id, account_id),
).fetchone()
```

这段也是教学简化。`execute`执行查询，`fetchone`读取一行。如果没有匹配行，得到`None`。生产入口使用`dict_row`，所以查询行可以用列名访问：`row["booked"]`。多行查询则可以用`fetchall()`读取结果集。

这就接上了Python：SQL挑出数据，驱动把结果交回Python，Python再做业务判断。驱动不会替你判断这个收款人是否正确。

3.3 INSERT与UPDATE：新增事实和更新投影

INSERT新增记录。购买会新增业务、请求结果、会计凭证和分录行。UPDATE修改允许修改的现有记录，例如增加累计借方，并把版本加1。

下面是`_post_transfer`中的源码节选：

SQL

```
UPDATE ledger.account_balance
SET debits=debits+%s,
    credits=credits+%s,
    version=version+1
WHERE workspace_id=%s AND account_id=%s
RETURNING 1
```

它做的是相对增加，不是把之前在Python里算出的旧余额直接覆盖回去。对客户扣80时，借方增量为80、贷方增量为0；对商户加80时，借方增量为0、贷方增量为80。

`RETURNING 1`让修改到的行返回一个结果。随后`fetchone()`如果得到`None`，说明没有更新到预期行，代码会报内部一致性错误。这里的1只是“确实更新到了”的标记，不是余额。

你不会在真实实现中看到`SET booked=booked-80`。因为booked是生成列，要改的是累计借贷。

3.4 一开始看不懂整条JOIN也没关系

真实的余额加锁查询还会把账户表和余额表JOIN起来。JOIN按共同的工作区和账户ID把两种信息拼到同一查询结果：账户属于谁、是什么类型，以及当前余额是多少。

这样Python拿到一行时，就能同时检查账户归属、币种、启用状态和可用余额。后面看到`FOR UPDATE OF b`，先把它理解为“把这次读取涉及的余额行锁住”；它如何解决并发，我们在第六章单独展开。

第四章：一笔购买怎样完整地走过Python和数据库

4.1 输入不是几串随意的字符串

购买入口接收三个重要对象：`Principal`说明可信调用者是谁；`PurchaseCommand`说明这笔业务想做什么；`RequestKey`说明这次逻辑请求用什么身份重试。

`Principal`由可信认证边界提供，包含工作区、客户端、业务主体。UUID本身不证明权限。知道某个购买ID，不能因此读取别人的结果或扣别人的钱。项目核心会校验账户归属；实际登录认证仍是调用方的责任。

`PurchaseCommand`包含业务ID、客户账户、商户账户、金额和币种。金额按正整数的规范字符串解析，再进入`AmountMinor`。`"80"`可以；小数、负数、浮点数或布尔值不能混成金额。Python里bool是int的子类，所以校验还专门避免把True当作1。

4.2 连接、游标、事务是三件事

连接`conn`是一条到PostgreSQL的会话。游标`cur`负责发SQL、读结果。事务把一组SQL组成一次整体提交或整体回滚。

连接可以先后执行很多个事务。关闭游标，不等于自动确认业务成功；代码里构造出一个成功对象，也不等于数据库已经提交。

`_submit_once`要求传入连接空闲，并且`autocommit=True`。这不是说“购买里的每条SQL各自提交”。它要求调用前没有隐藏的外层事务，随后由入口明确开启自己的事务。

核心边界的源码节选如下，中间业务步骤已用注释省略：

PYTHON

```
with conn.transaction(), conn.cursor(row_factory=dict_row) as cur:
    # 入口、身份认领、重放或新业务处理
    # 实际代码在这里决定 result
    ...
return result
```

这里的省略号是为了展示结构。进入事务上下文时开始事务；正常退出时提交；异常退出时回滚。`return result`在事务with退出后，因此提交时的延迟约束失败，会先抛异常，不会先把201交给调用方。

4.3 把真实执行顺序读成一个故事

客户端提交购买P1，请求键K1，客户账户给商户M1付80。`submit_purchase`进入有上限的重试包装，再调用一次完整尝试。

这一完整尝试先设置事务隔离级别和超时，然后拿工作区共享屏障、读取模式，确认当前允许写入，并检查客户归属。随后先认领请求键，再认领业务ID。若已经有原结果，就进入重放；若是新业务，才锁账户余额并判断能否购买。

在本次全新购买中，客户100，商户0。判断通过后，代码新增凭证和两条分录，把客户累计借方加80、商户累计贷方加80，并保存业务结果与请求结果。最后退出事务with，数据库执行提交检查，提交成功，返回201。

推演 / 结构

```
进入事务
  → 共享屏障与模式/授权检查
  → 请求键认领
  → 业务ID认领
  → 余额行锁与余额判断
  → 凭证、分录、投影、两份结果写入
提交成功
  → 返回201
```

这是根据源码整理的教学路线，不是一次运行的SQL日志。真实运行可能走重放或拒绝分支；学习工具的SQL追踪记录那次实际执行的命令。

4.4 事务到底保证了什么

正确代码把这笔业务的所有写入放进一个事务。如果中间SQL失败，已经写入但尚未提交的部分会整体回滚，不会只提交客户那一侧。

但事务只能一起提交你实际写进去的内容。假如程序漏写了商户更新，或同时把分录和余额记给错误商户，原子提交不会凭空知道你漏了什么。因此，事务原子性、业务逻辑正确性、独立核对，是不同层的保证。

现在购买完成：客户20，商户80。接下来，最值得研究的不是再写一个新功能，而是把完全相同的请求再送一次。

第五章：请求重来一次，为什么不会再扣80

5.1 两个身份分别回答两个问题

业务ID回答“这是哪一笔业务”，例如购买P1。请求键回答“这是某个调用方、某条路由上的哪次逻辑请求”，例如K1。

两者都有作用域：业务身份是`(workspace_id, operation_id)`；请求身份是`(workspace_id, client_id, route_family, key)`。购买和冲正有各自的路由族，不是把一串key放到全局就算完成幂等。

`operation`保存业务的判定；`request_result`保存请求与业务及结果的关联。一个业务可以在原内容不变、授权通过的前提下关联另一个请求键，但资金效果仍然只有原来那一份。

5.2 先看三次不同的“再来一次”

第一笔P1/K1已经成功购买80，客户20、商户80。接下来的三种情况不要混为一谈。

再次发送什么	业务ID	请求键	实际含义和结果
原内容原身份	P1	K1	重放原201，不新增购买分录
同一业务换请求键，内容仍相同	P1	K-alt	关联新请求键并重放原201，无新资金效果
同一请求键，把金额80改成70	P1	K1	内容冲突409，回滚，不重新买70

如果用K1却换成新的P2，即使金额仍是80，也已经更换了请求内容。业务ID本身就是内容的一部分。想发起另一笔购买，需要同时正确使用新的业务身份和请求身份。

这张表是三个独立对照，不是接着主线全部执行一遍。主线接下来使用新的P2/K2购买，因此K2尚未被占用。

这些表格都以工作区允许受理、调用者有权访问为前提。真实入口先检查模式和授权，再进入重放；FROZEN期间，即使原身份曾经成功，也会先得到503，不能把“重放稳定”讲成“绕过一切准入条件”。

5.3 系统比较的“原内容”是什么

项目把业务字段整理成固定版本、固定表达方式的规范化字节，再保存到`operation.payload`等记录。它包含工作区、业务ID、账户、金额等业务内容。

这不是原始HTTP报文的逐字备份。HTTP头、登录凭证、追踪时间戳和请求键，不属于这份规范化业务内容。代码也保存SHA-256摘要辅助标识，但不能把“摘要看起来一样”当成不再校验内容的理由；当前重放会比较保留的规范化字节及相关结果。

规范化的目的，是让“同一份业务内容”有明确、可比较的表达。否则客户端仅仅改变字段排列，就可能被误判成另一笔业务；反过来，粗糙转换也可能把不同值错误地当成一样。

5.4 两个相同请求同时来，谁先认领

源码中的请求认领使用了类似下面的结构。这是教学简化，实际SQL包含全部身份、payload和摘要字段。

SQL

```
INSERT INTO ledger.request_result (...)
VALUES (...)
ON CONFLICT ON CONSTRAINT request_result_pkey
DO NOTHING
RETURNING 1;
```

这条语句不是“先SELECT看看有没有，再凭感觉INSERT”。主键唯一性由数据库协调。如果另一个未结束事务正占着相同键，这次认领会等它结束。

先到的事务提交后，后到者会发现自己没有认领到新键，随后读取已保留记录，校验授权和内容，再重放或报冲突。数据库唯一性处理竞争，Python解释这个竞争结果在业务上意味着什么。

同业务ID的竞争也采用相应的认领和重放流程。顺序固定为先请求键、再业务ID，让读者能明确看到多个请求在哪一步可能等待。

5.5 余额不足，也要把结果保存下来

回到主线：P1买80成功后，客户只剩20。现在用新身份P2/K2再买80，得到422 `INSUFFICIENT_AVAILABLE`。

这一次没有新增购买凭证，没有修改余额，但会保存业务拒绝和请求结果，然后正常COMMIT。

“业务被拒绝”与“数据库事务失败”是两件事。余额不足是这次业务的确定判定，所以提交这份判定；能够确认未提交的事务异常、身份冲突或提交检查失败，会按相应异常路径回滚；COMMIT期间断线是否已经提交，另见下一节的未知结果恢复。

后来客户资金恢复，重新发送P2/K2仍应返回原来的422。否则一次响应丢失后的重试，可能把已经拒绝的业务改成成功。要基于新的资金情况再做一次判定，就必须发起新的业务。

并非所有错误都会保留：无权访问的404、身份冲突409、冻结或争用返回的503，不会被混成一条已保存的422。这些错误的恢复方式也不同。

5.6 最危险的是“我不知道它到底提交了没有”

设想数据库已经COMMIT，但响应在回到客户端之前丢失。客户端看见网络错误，无法从这个错误判断“没入账”。如果这时生成新业务ID再买一次，就可能形成两次合法购买，系统没有理由把它们当成重试。

正确恢复方式是保留原业务ID、原请求键和原内容，在调用方重新连接后再次提交。若第一次已提交，读取保存结果；若第一次没有提交，原身份可以完成那一次业务。

项目核心会把连接断开等错误传播给调用方，没有悄悄自动重连。它也没有宣称网络只会投递一次。它提供的是：在当前身份契约和受控入口下，用持久化结果恢复同一笔业务，而非把每次网络尝试都当成新购买。

最后把三个词并排放好：重放是读取已提交判定；事务重试是重做一个明确失败的事务；新业务是请求系统重新做一次业务判定。下一章会讲第二种。

第六章：两笔80同时花100，事务为什么还不够

6.1 这个例子从新的100重新开始

这一章单独重开一个工作区：客户100，商户0。购买A与购买B是不同的业务ID和请求键，各自要付80，所以幂等机制不能把它们合并成一笔。它们确实是两笔独立购买。

如果只是“先读100，觉得够，再各扣80”，两个请求可能都作出余额足够的判断。把每个请求各自装进事务，也不自动保证这两个判断之间形成正确关系。

我们需要让同一余额的“读、判断、更新”受到协调。当前实现选择先取余额行锁，再使用读到的行判断。

6.2 FOR UPDATE锁的是行，不是整个数据库

`_lock_balances`的真实查询会JOIN账户信息和余额，并以`FOR UPDATE OF b`锁住余额表别名b对应的行。

理解上可以先看这个教学简化：

SQL

```
SELECT booked, reserved
FROM ledger.account_balance
WHERE workspace_id=%s AND account_id=%s
FOR UPDATE;
```

购买A拿到客户余额行的锁后，购买B也想以冲突方式锁同一行，就必须等。A会一直持有事务中的行锁，直到提交或回滚。

普通只读查询并不等同于这条加锁查询。不要把“余额行被锁住”理解为全库都不能读、所有账户都不能交易。

6.3 READ COMMITTED下，等待者拿到什么

当前业务入口使用READ COMMITTED隔离级别。理解这次竞争时，抓住这一条：等待余额行锁的查询，在前一个修改者提交后，会取得更新后的目标行，再据此判断。

顺序	购买A	购买B	已提交客户余额
1	取得客户余额行锁，读到100	到达同一行，等待锁	100
2	判断可付80，写入客户20和商户80	仍等待	100
3	COMMIT，释放锁	可以继续取得行锁	20
4	已完成201	读到20，不够付80，保存422并提交	20

第二行的“已提交余额仍是100”是因为A的修改尚未提交，不能把A事务内的20直接当成其他会话已看到的状态。

这条时间线展示一种顺序；关键结果是只有一笔成功、另一笔被保存为余额不足。不要把READ COMMITTED理解成“事务从开始到结束都固定看同一快照”，也不要说“等锁以后整个事务会自动重读所有表”。这里说的是相关加锁语句取得更新后的目标行。

6.4 为什么要按账户UUID排序

一次购买涉及客户和商户两个余额。一个事务若先锁客户再锁商户，另一个却先锁商户再锁客户，就可能形成循环等待：你手里有我需要的锁，我手里有你需要的锁。

`_lock_balances`按账户UUID的数值顺序锁定，而不是一律按“客户在前、商户在后”。相同的一组账户总有相同顺序，可以消除这些余额行上的相反顺序循环。

这个结论有范围：它约束当前实现的加锁路径，不证明未来任何批处理、其他表或任意手写SQL都不会死锁。冲正还要先锁原购买，因此也必须审查它与其他路径的整体顺序。

6.5 为什么有了Python判断，还保留数据库CHECK

Python给出清楚的业务拒绝：余额不足就保存422。数据库的非负和预留关系约束提供另一道防线：即使程序某处把判断写错，也不能提交违反这些值域规则的余额。

但两道防线都不能独自证明收款商户正确，也不能证明分录与余额一致。它们分别保护明确的性质，不是相互替代。

也可以研究条件UPDATE或更强的隔离级别，但那是另一组实现取舍。当前项目的面试重点，是能解释已经选择的行锁顺序、判断位置和提交边界。

6.6 什么失败可以自动重试

`_submit_with_retry`只对两类明确允许的数据库失败重做整笔事务：40P01死锁、40001序列化失败。最多三次总尝试，也就是最初一次加至多两次重试。

重试保留原业务身份和请求身份，必须在连接仍打开且回到空闲状态后进行。不能从“扣款那条UPDATE”单独继续，而是重新执行整个事务逻辑。两次重试前的退避包含随机等待，当前范围分别是10至50毫秒、20至100毫秒。

超时、取消、连接断开、普通完整性错误，不会被一概放进自动重试。提交结果未知更不能简单换ID重发。

40001处理分支在真实PostgreSQL测试里通过服务端故障注入验证。当前购买使用READ COMMITTED，不能把这项测试讲成“系统自然在SERIALIZABLE隔离级别产生了冲突”。

6.7 怎样证明它们真的同时等待过

学习实验先用一条独立连接持有客户余额行锁，让A、B真的进入等待。另一条观察连接查询`pg_stat_activity`和`pg_blocking_pids`，确认谁在等谁、当时执行什么SQL，再允许放行。

这种阻塞者用于把瞬间过程停下来供人阅读，不是购买算法的一部分。观察到的等待关系才是证据，不能用“等了半秒”代替。

理解原理不依赖这套页面。你已经通过本章的100、80和锁等待时间线，走完了同一件事。

第七章：冲正怎样把钱还回去，同时保留历史

7.1 回到主线：客户20，商户80

这一章接回第五章主线。P1购买80成功，P2再次购买80被拒，余额仍是客户20、商户80。

现在原商户发起一笔新的全额冲正V1。金额和账户来自经过校验的原购买记录，不是让调用方随意指定“给谁退多少”。当前R2只支持成功购买的全额内部冲正。

冲正新增一张凭证：D商户80、C客户80。原购买的凭证和分录不修改、不删除。

对象	购买后	冲正新增什么	冲正后
客户	D80 / C100，余额20	贷方累计加80	D80 / C180，余额100
商户M1	D0 / C80，余额80	借方累计加80	D80 / C80，余额0
清算资产	余额100	无新的清算资产分录	余额100

你可以同时解释两件事：余额恢复了；购买曾经发生的事实也还在。这正是追加反向分录比直接删除原购买更适合追溯的原因。

7.2 为什么由原商户发起

如果客户只凭购买ID就能扣商户的钱，接口就承诺了客户可以单方面退款。R2没有作出这个承诺，而是要求原购买的商户主体发起。

代码通过可信`Principal`检查资源和归属。原购买不存在、没有成功，或调用者不是原商户时，统一返回404，避免泄露对象是否存在。原客户账户停用时，新冲正也可能返回404且不保存422；不能把“每次业务拒绝保存”扩大为“任何错误都永久留下冲正尝试”。

7.3 为什么需要两张冲正关系表

系统需要同时满足两个要求：失败尝试可以保留很多次；同一原购买最多只有一次成功冲正。如果把所有尝试都按原购买ID设置一个唯一键，一次余额不足的失败就可能占住名额，挡住后来合法的成功。

因此它用两张关系表达不同事实：

表	关键身份	表示什么
`reversal_attempt`	工作区 + 本次冲正业务ID	这次冲正尝试针对哪笔原购买
`purchase_reversal_effect`	工作区 + 原购买ID	哪一次冲正已经成功形成效果

成功关系只在成功时写入。失败尝试不会占住它。规则例子是：商户可用余额不足，第一次冲正被拒；资金条件满足后，用新业务ID再次尝试可以成功；重放第一次的业务ID仍返回原拒绝。这是独立规则示例，主线中的V1已经成功，不能再对P1形成第二次成功。

7.4 两个冲正同时来，为什么不会都成功

新冲正先锁原购买那一行，再按账户UUID顺序锁余额。同一原购买的两个冲正会在原购买锁上排队。

前一笔成功提交后，后一笔取得锁，再查询成功关系，就能看到它已经存在，于是保存422 `ALREADY_REVERSED`。成功关系的主键还提供数据库层的一次成功限制。

如果正常锁约定之外仍触发未预期的唯一键异常，代码不会一概吞成“已冲正”，因为那可能是实现约定被绕过的信号。

7.5 重放成功冲正，与新业务重复冲正

用V1原身份、原内容重放，仍返回保存的201。生成新冲正V2去冲正同一个P1，得到保存的422。两者都不会再增加分录，但它们不是同一件事。

主线此时是客户100、商户0。下一章故意把客户投影改成120，看看哪些检查会通过、哪些必须失败。

第八章：账都平了，为什么还可能把钱记错

8.1 第一个反例：只把余额投影加20

主线中客户累计借方80、贷方180，余额100。故障注入使用临时库所有者连接，只把客户投影的`credits`加20：

推演 / 结构

```
分录重算：credits180 − debits80 = 100
错误投影：credits200 − debits80 = 120
```

分录没改，所以原来每张凭证依然借贷平衡。投影的生成列也没违反公式，120同样不小于预留0。仅看这些规则，错误不一定被拒绝。

这就是核对A要发现的错误：根据全部分录重新计算每个账户的累计借贷和余额，再与投影逐项比较。没有交易的账户也要算进去，不能因为它没有分录就从检查里消失。

本例A应报告`PROJECTION_DRIFT`，预期100，实际120。数据库约束和A不重复：前者保护声明过的结构和值域，后者独立验证跨表推导关系。

8.2 第二个反例：分录和投影一起记给错误商户

另设一个反例：原请求明确要付M1，代码却把分录与余额都正确地配套写给M2。客户少80，M2多80；凭证平衡，投影也完全符合这些错误分录。

A这时可能通过，因为它只回答“余额是否符合分录”。它不能单凭这些分录知道业务原来想付给谁。

核对B独立解析保存的规范化购买请求，恢复预期客户、商户、金额和方向，再与实际效果比较。请求写M1，分录写M2，它就能报`WRONG_OWNER`。金额不符则有相应的金额错误。

错误	借贷平衡可能通过吗	A可能通过吗	主要由谁发现
只错改余额投影	可以	不可以	A：分录与投影不一致
分录、投影一起记错商户	可以	可以	B：效果不符合保存的请求

B的权威来源是记账前保留的业务请求，不是从当前分录反推“它本来想干什么”。否则错误分录会自己给自己作证。

8.3 B也有明确的能力边界

如果最初保存的请求本身就错成了M2，B只能证明分录与这份已存请求相符，不能凭空恢复用户真正想付M1的意图。测试里额外使用手工冻结的预期指令，可以检查这类请求捕获错误。

实际本地业务核对不要求人事先写一张手工清单，所以正常运行的数据也能导出核对、进入符合条件的重建。

合成OPENING没有与购买同样的规范化入金请求契约。核对器可以检查结构，但不能证明它是真实银行入金，报告会披露这类意图尚未核实。

历史余额不足也不能用今天的余额重新裁判。核对器检查保存的拒绝没有误产生财务效果，不会因为现在有钱，就反过来说过去的422一定错误。

8.4 为什么不能每张表随便导出一次

假设先导出业务表，然后一笔购买提交，最后再导出余额表。这个导出包可能拼出一个数据库从未同时存在过的状态：业务还是旧的，余额却已经变了。独立核对就会产生误报。

`capture_snapshot`使用一个REPEATABLE READ、READ ONLY事务读所有相关表。它们共享同一事务视图；快照在相关查询开始时建立，不要把它机械地说成BEGIN的墙钟时刻。

这不是冻结整个数据库。其他正常事务仍可以提交，导出事务依照自身的一致视图读取。购买的READ COMMITTED与导出的REPEATABLE READ解决不同问题，不必因为名字不同就认为其中一个选错。

导出完成并关闭数据库连接以后，离线核对器只读本地完成的文件。文件摘要检查用于发现损坏或内容不符；业务是否记对，仍要由结构、A、B这些语义检查回答。

8.5 为什么独立核对不直接复用记账函数

假如记账函数把商户方向写反，核对又调用同一个函数“计算预期”，双方就可能共享同一错误，最后错误地相互认可。

`verification.py`使用标准库独立解析和计算，不导入生产记账步骤。测试中的参考折叠和手工预期也承担类似职责。独立不意味着绝不会写错，但它避免了最直接的同源错误。

看报告时分开读：`structure_ok`说明结构检查通过；`projection_ok`对应A；`request_intent_ok`对应B。`healthy`组合这三项。`reservation_supported`另行参与重建准入，不属于healthy的组合条件。

所以一句“核对健康”有明确含义：支持范围内的结构、投影、保存请求意图相符。它不是生产付款完成、原请求真实或所有预留都可重建的证明。

第九章：把120修回100，为什么要先冻结

9.1 现在错的是投影，不是分录来源

回到主线，客户投影120，但分录算出100。结构和B通过，A失败。这类错误可以通过可信来源重新计算投影来修。

如果是记错商户、B失败，仅仅照旧分录重算余额，只会更忠实地把钱继续算给错误商户。重建不负责把错误业务来源变成正确业务。

这就是维护准入首先要区分“来源不可信”与“投影偏离来源”的原因。

9.2 只改mode字段，为什么叫不上排空

设想一笔购买已经进入，读到NORMAL，随后卡在余额锁；这时维护者直接把mode改成FROZEN。旧购买仍可能继续提交，维护窗口并不稳定。

当前协议让业务写事务先拿共享的事务级advisory lock，再读mode。维护事务拿同一工作区键的排他锁。共享与共享可以并存，共享与排他互斥。

顺序	在途购买	冻结维护
1	持有共享屏障，继续业务	请求排他屏障，等待
2	COMMIT或ROLLBACK，释放共享屏障	终于取得排他屏障
3	已结束	把mode写成FROZEN并提交
4	后来的新购买拿共享屏障，再读到FROZEN，返回503	已完成冻结

排他屏障等到在途协作写事务全部结束，这叫排空。FROZEN是排空后保存的工作区模式。锁先于模式检查，才把两个时点连接起来。

advisory lock是约定式锁，只约束主动遵守同一协议的代码。数据库所有者仍可以执行不遵守协议的SQL。数据库冻结也不代表外部付款停止；当前R2没有接外部支付渠道，更没有取消第三方请求的能力。

9.3 预览让你知道差异，但不提供待回写的数字

冻结后，预览导出当前状态并计算差异。本例会看到客户120，而分录目标是100。

真正应用重建时，`apply_rebuild`在排他屏障下重新读取数据库、检查来源，再用SQL从分录现场聚合目标。它不把旧预览文件里的100直接复制进余额表。

预览ID只是调用方提供的审计引用，不证明系统认证过这个文件，更不能证明某个人真的看过预览。这个边界写清楚，才不会让一个普通UUID承担它没有实现的审批意义。

当前规模适合在一个事务内现场重算，因此没有候选余额表和候选激活生命周期。未来若必须离线算很大的数据，再重新设计一致性与激活契约。

9.4 一次修复有哪些东西一起提交

应用事务要一起完成：重新计算并写入余额投影；记录每个账户改前、改后的值；更新账户行版本；把工作区`projection_epoch`从1加到2；保存修复元数据。

主线中的客户从D80/C200修回D80/C180，于是生成余额从120回到100。不会直接给booked赋值，也不修改原购买或冲正分录。

当前实现重算并写入全部账户，包括数值没有变化的账户，行版本也会相应增加。因此`version`不是“这个账户买过几次”，`projection_epoch`也不是交易数量。前者标记余额行更新，后者标记整个工作区的投影重建代次。

SQL计算写完后，同一个事务内还会再次调用独立Python核对器。如果写后结果不一致，余额、修复明细和epoch整体回滚。测试曾把旧预览目标替进应用路径，确认这种漂移会被写后核对发现。

修复记录同样要防止事后修改、删除和追加明细。否则历史上“这次究竟修过什么”就会被改写。

9.5 重建和恢复营业，是两道不同的门

这两个条件最容易混，所以直接并排看。

动作	要求结构正确	要求A通过	要求B通过	要求预留为0
应用重建	是	旧A不必通过	是	是
恢复NORMAL	是	是	不作为阻断条件	不作为阻断条件

重建不要求旧A通过，因为修复A发现的投影漂移正是它的任务。它要求B通过和零预留，避免用错误来源修复，也避免把尚未支持重建的预留生命周期悄悄清零。

恢复营业只要求结构和A通过；B及预留情况仍写入报告和恢复记录。这个选择不意味着B问题被解决，而是把“结束投影维护”与“纠正业务来源”分开处理。

否则可能出现死循环：历史记错商户使B一直失败；原分录不能改；冲正又要求NORMAL；若恢复必须先让B通过，就变成要纠正才能恢复、要恢复才能纠正。

重建提交后仍然FROZEN，恢复是下一次明确调用，并会现场重新核对。冻结、修复等也有自己的稳定身份，提交确认丢失时应按对应身份恢复结果，不能随意重复制造维护动作。

9.6 把主线十二步的结果放在一起

读到这里，整条链已经完整。下面只是把已经解释过的数字集中一次，便于回看。

步骤	发生什么	客户	商户M1	关键结果
1	合成开户	100	0	清算资产100
2	P1购买80	20	80	保存201，新凭证
3	P2再买80	20	80	保存422，无新凭证
4	V1冲正P1	100	0	保存201，新增反向凭证
5	V2再次冲正P1	100	0	保存422 ALREADY_REVERSED
6	仅把投影credits多加20	120	0	分录未改
7	导出并离线核对	120	0	结构/B通过，A失败
8	冻结并排空	120	0	FROZEN
9	预览	120	0	目标客户100
10	事务内现场重建	100	0	epoch变2，仍冻结
11	重新核对后恢复	100	0	NORMAL
12	新导出并离线核对	100	0	结构/A/B通过

这条标准故事最终保留5条业务、4条请求、3张会计凭证和6条分录行；冲正尝试2条，成功关系1条。3张凭证分别是开户、成功购买、成功冲正。被拒绝的业务有记录，但没有购买或冲正凭证。

第十章：现在去读代码，应该先看哪几段

10.1 从目录入口读，不要从第一行读到最后一行

先打开`src/ledgerops/db/admission.py`，找`submit_purchase`。它只是把调用交给`_submit_with_retry`。后者告诉你：一次尝试失败，哪些错误才允许从头再来。

再找`_submit_once`。把它当目录：检查输入与连接状态，进入事务，调用`_open_admission`，先认领请求，再认领业务，最后分到重放或新业务。先读懂分支，再钻进每个函数。

新购买分支是`_post_new_purchase`：取得余额，检查归属，判断拒绝原因；若通过，调用`_post_transfer`；最后保存业务和请求结果。`_lock_balances`负责按序加锁，`_transfer_rejection`负责余额和边界判断，`_post_transfer`负责具体会计与投影写入。

冲正也通过同一个事务入口和重放/重试包装，差异在`_post_new_reversal`。它先锁原购买，记录尝试，检查成功关系，再以反方向调用转账步骤。

这种阅读顺序比一次吞下整个文件更接近业务的实际调用路径。文件里其他函数，等你在调用点遇到时再读。

10.2 你会遇到的Python写法，现在怎么解释

`@dataclass`给数据对象生成初始化等常用方法。`frozen=True`阻止普通字段重赋值，帮助表达“这个输入对象不应在重试中被随便改写”。它不是数据库不可变保证，也不自动让嵌套对象全部深度不可变。

类型标注说明预期输入；项目还在关键入口用运行时检查验证对象类型。类型标注本身不会让任何错误输入自动消失。

`raise`表示抛异常，沿调用栈退出；若发生在事务with内，会走回滚。`return AdmissionResult(422, ...)`则可以作为正常业务结果保存并提交。区别不在数字是否小于400，而在这条代码路径表达的是确定判定还是异常失败。

`Jsonb`把Python对象交给驱动，按PostgreSQL JSONB类型保存。`uuid4()`生成新身份。重试同一业务时，业务ID和请求键由调用方保留，不应在每次尝试里重新生成成另一笔业务。凭证ID则在新入账分支生成；失败事务里的新凭证不会单独提交。

`@contextmanager`用于维护模块的`_transaction`：yield之前设置事务、屏障和工作区读取，调用者在yield提供的上下文中完成动作，退出后统一提交或回滚。你已经在第四章见过同一个事务边界思想。

10.3 测试数量背后，应该会解释哪些证据

这个代码版本全量609项测试通过，这是本地既有验收事实；这份教程是文档交付，没有为修改文字重新声称跑了一轮财务回归。

数量不等于理解。面试时更有用的是解释：一个测试主动制造什么错误，它应该由哪一层发现。

想发现的关键错误	检验方式	相关位置
两笔80都成功	多连接真实锁竞争，检查201/422与字面余额	`test_purchase_concurrency.py`
原键换内容被当成新购买	实际409、回滚、财务行不变	`test_purchase_admission.py`等
提交后响应丢失导致重扣	实际进程/连接故障，再用原身份恢复	`test_purchase_recovery.py`
漏更新余额、记错商户	独立A/B和手工预期分别失败	`test_independent_checks.py`
使用过时预览直接修复	变异写入后独立检查，整个维护事务回滚	`test_maintenance.py`
失败冲正挡住以后成功	保留失败，再用新身份满足条件后成功	`test_reversal_admission.py`
取消演示时误提交冻结	先取消依赖者，确认无冻结记录、连接清理	`test_lab_concurrency.py`

这些文件位于`tests/integration/`。它们使用真实PostgreSQL与实际角色；SQLite或纯mock不能证明PostgreSQL行锁、延迟提交检查和这些权限关系。

金额解析、身份格式等局部规则有单元测试。`reference/`和独立核对器提供不同算法的对照。故障注入和变异测试的价值，是尝试让系统犯一个已知错误，看证据是否真的能抓住它。

10.4 技术工具各自解决什么

Python表达业务流程；psycopg连接数据库并执行参数化SQL；PostgreSQL提供事务、行锁、约束和持久化；Docker给本地验收创建可清理的真实数据库环境。

pytest组织测试，Ruff检查代码风格及部分静态问题，uv管理锁定的Python环境，Git保存可定位的本地提交。导出/核对CLI便于不依赖页面验证文件；学习HTTP页面帮助观察，但不属于正式生产接口。

没有用Redis并不意味着幂等不存在：当前幂等与财务写入在同一个PostgreSQL事务中完成。没有引入消息队列，也就不能声称项目已经处理了真实外部付款消息投递。理解已有组件之间的责任，比为每个流行名词补一个组件更重要。

10.5 把项目组织成一段面试说明

可以按“问题、机制、失败证据、边界”组织，而不是先报技术栈。

“这个项目实现平台内部账本。购买在Python显式事务里完成，保存请求和业务身份，原内容重试返回原结果。两个独立扣款通过按序余额行锁协调，因此不能各自花掉同一份余额。数据库约束保护已声明的结构和值域，但我不会用借贷平衡代替业务正确性证明，所以另有独立A/B核对：A查投影，B查保存请求中的收款对象。R2还能冻结排空后现场重建投影，并用反向分录完成全额冲正。”

接着补一项具体证据：“例如两笔80争100，实际等待关系由PostgreSQL报告，最后一笔201、一笔422；或故意把投影100改为120，A失败，维护后修复成100，并保留修复日志。”

最后说清限制：“当前是本地合成资金实验，没有生产登录、真实银行渠道或容量承诺。数据库冻结只约束遵守屏障的写路径，不证明外部付款已停止。”

回到具体函数和失败例子，这段说明就有了依据；遇到继续追问时，沿着下面的附录展开。

附录A：面试继续往下追时，怎样展开

A.1 有事务，为什么还要锁

事务解决一笔业务的原子提交；锁协调多笔业务对同一资源的竞争。两笔事务各自完整，并不自动保证它们都基于有效余额作判断。当前实现将加锁、读取当前行、判断和修改放在同一事务内。

A.2 有非负CHECK，为什么还要Python判断

CHECK提供拒绝违规状态的数据库防线；Python判断提供业务可解释、可持久化的422。若只依赖数据库报完整性异常，就不能等同于已保存一次余额不足判定。两层还都不能证明商户归属或投影一致，需要相应业务检查和独立核对。

A.3 为什么不把全部逻辑写进数据库函数

当前选择Python主导流程、显式SQL，便于直接审查锁、重放和测试路径；数据库保护结构与不可变历史。代价是信任运行代码按协议写入，普通运行账户仍能写出某些数据库合法而业务错误的状态。数据库函数收入口是另一种取舍，不能把当前方案描述成拥有它没有实现的不可绕过入口。

A.4 不允许UPDATE/DELETE，为什么还要防追加

如果已提交凭证允许再加一借一贷，原有行都没改、借贷仍平衡，但凭证整体已经被改写。当前表头不可改，并声明行数和单边总额；延迟约束检查最终明细与声明，所以这种追加也会失败。数据库所有者能修改保护机制，仍在普通运行角色保证之外。

A.5 为什么不用更强隔离级别解决一切

更强隔离级别也有语义、重试和吞吐代价，还不能替代业务身份、授权和正确收款人检查。当前选择READ COMMITTED加明确行锁，已有真实并发证据支持这些受控路径。若换实现，要重新证明相同性质，不能只把隔离级别名字改掉。

A.6 为什么不知道COMMIT结果时必须保留原ID

网络错误可能发生在数据库提交之后。新ID会成为另一笔业务，唯一约束没有理由把它拒绝为重复。原身份恢复把“这次网络尝试”重新连接到“原来那笔业务”，读取已有结果或完成尚未提交的那次业务。

A.7 为什么恢复允许B失败，重建却不允许

重建会用分录来源重新写投影，B失败说明来源可能表达了错误业务，不能直接依赖它。恢复只结束这次投影维护，要求结构与A正确，并把B告警留下；它不会把业务错误洗成健康。两种动作的责任不同，条件不能机械复用。

A.8 这些测试能不能证明所有并发都安全

不能。它们证明已覆盖的输入、故障位置与调度满足约定。固定锁顺序、唯一约束和事务协议提供可分析的依据，具体测试提供可复现的反例检验；未来新增写路径、批处理或外部系统时，还要重新检查锁依赖、身份和恢复边界。

附录B：源码地图与几个容易忘的词

想继续读什么	直接找哪里
金额为什么用精确整数	`src/ledgerops/domain/money.py`
业务请求怎样规范化	`src/ledgerops/domain/requests.py`
购买、冲正、重放与重试	`src/ledgerops/db/admission.py`

想继续读什么	直接找哪里
表、约束、权限与历史保护	`src/ledgerops/db/sql/001_foundation.sql`及后续迁移
导出快照与格式兼容	`src/ledgerops/export.py`
独立结构/A/B核对	`src/ledgerops/verification.py`
冻结、预览、现场修复、恢复	`src/ledgerops/db/maintenance.py`
学习工具的调度与真实等待观察	`lab/concurrency.py`

幂等：在当前身份契约下，重复提交同一业务不会新增资金效果，并能重放原结果。它不代表所有重复网络投递都被物理消除。

投影：从历史事实推导出来、为了读取方便而保存的当前表示。投影可以重建，但前提是来源可信且当前支持这类状态的重建。

不变量：系统宣称始终应成立的条件。必须逐条写明由哪一层保证，或由哪种核对发现偏离，不能只说“数据库兜底”。

SQLSTATE：数据库错误码，例如40P01、40001。它与HTTP语义状态201、422、409、503不是同一套东西，不能混着判断恢复方式。

快照：事务按隔离规则看到的数据视图。导出文件保存的是这份观察，不会随着数据库后续变化自动更新。

epoch与version：前者是工作区投影重建代次，后者是余额行更新版本。它们都不是金额、权限或交易次数。

回到最初那笔购买：业务请求带身份进入Python；Python在一个显式事务里按顺序检查和写账；数据库落实锁与声明过的规则；独立程序再检查保存事实之间的关系；发现投影漂移后，用受控维护和完整记录恢复。每一层都解决具体问题，也有明确边界。